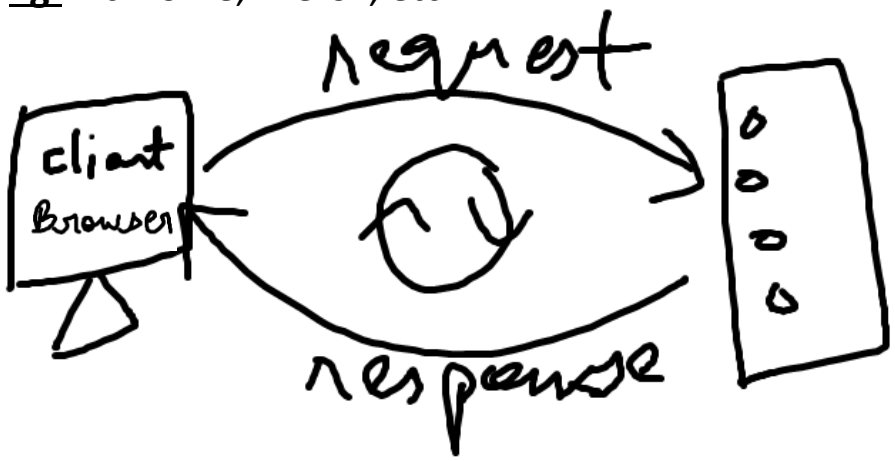
Java Script

Javascript is the only programming language understood by our browser.

Programming language – Any language that can be used to create logics.

Browser – where we can send request to server and get the response.

Eg. – chrome, firefox, etc



Primitive – smallest possible division, which cannot be further broken down. (Building block of your program)

How many primitive datatypes in javascript?

- **Number** (int & float)
- **Boolean** (true & false)
- **String** (char & string)
- **Undefined**
- **Null**
- **Symbol**
- **Big int**

Variable – names which hold some values or points to some value. It is not a container in JS

Eg. – Let age = 23;

Let umar = Akmal;

Declarative – with the help of which we can define the variables.

- var
- let
- const

Nomenclature

Way of declaring the variables… (Variables ko kon kon se naam de sakte ho)

- can contain (a-z), (A – Z), (0,1, 2,… 9)
- can contain _ , $
- must not start with a number.
- must not have predefined words.
- can be start with _ , $

Eg. – let mann = 10; ✓

let marziya = 20; ✓

let mann ✓

let 2mann ✗

let man_made ✓

let man$made ✓

let _ ✓

let $ ✓

let break ✗

Way of defining the variables

- **Normal case**

  Eg. – let venugopaliyer = “goa”;
- **Camel case**

  Eg. – let venuGopalIyer = “Readability”;
  - Most preferred case.
- **Snake case**

  Eg. – let venu_gopal_iyer = “Saanp”;
- **Kabab case**

  Eg – let venu-gopal-iyer = “Bhaukaal”;
  - We cannot use kabab case in javascript because of the hyphen.

Note –

We will use camel case in our js.

Ques - What is a console?

Ans - Console is a **REPL**.

- **R** -> Read
- **E** -> Evaluate
- **P** -> Print
- **L** -> Loop

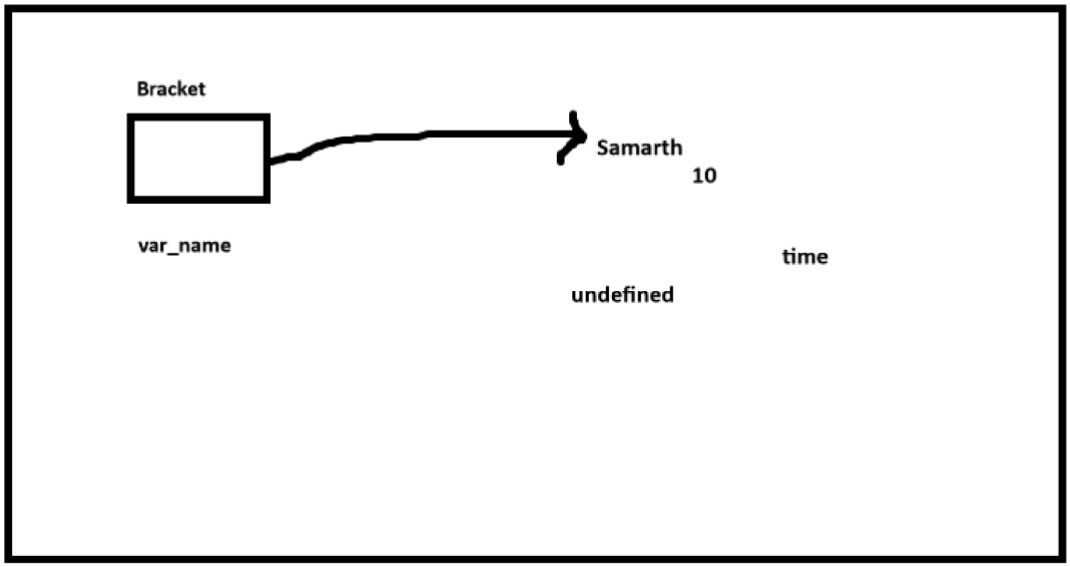
Interpreter	Compiler
->Line by line execution occur.	->First convert the code into executable code, then run the entire code all at once
->If there is an error in one line, then the next line will not be executed.	->If there is an error in any line, then the whole code will not be executed.

JS -> interpreted, weakly typed, dynamically typed, sungle threaded.

- **Interpreted –** line by line
- **Weakly typed –** where the data type(primitive) of the variable is not fixed.
  Eg. – let umar = 17;
  umar = “17 baras”;
- **Dynamically typed –** at run-time, data type(primitive) is defined.
  Eg. – let sam = 100;
  sam = “abcd”;
  sam = true;
  sam = undefined;
- **Single threaded –** Ek time par ek kaam hoga , i.e, multiple tasks cannot be performed at the same time.

String

1) Let var_name = “Samarth”;



2) let var_ka_naam = ‘hello’

Note – There is no difference between “”/’’, both are considered as same.

3) let name = `hello`;

`` → Backticks

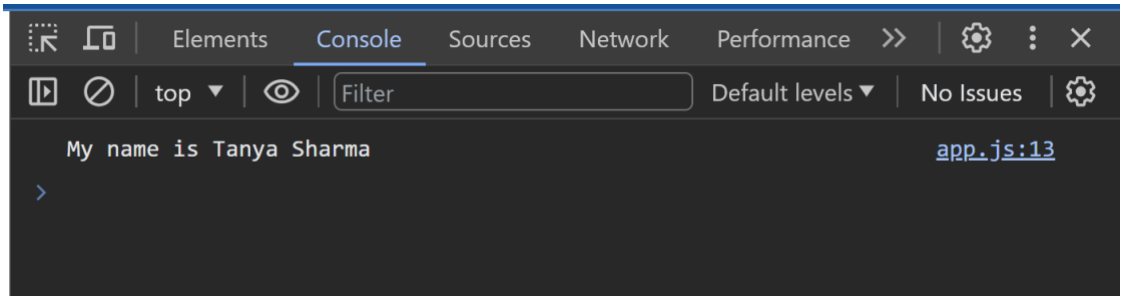
```
let string1 = "samarth 1"; let string2 = 'samarth 2';
let string3 = `samarth 3`; // \advantages
```

Semicolon is not necessary to use in JS. It only terminates the lines if there are multiple instructions given in a single line.

Concatenation (+)

It joins or adds the multiple strings together and print them as a single string.

```
let var1 = "My name is ";
let var2 = "Tanya ";
let var3 = "Sharma ";

// concatenation(+)

let ans = var1 + var2 + var3;
console.log(ans);
```



Direct strings can be appended too.

```
let ans = var1 + ' ' + var2  + ' ' + var3;
console.log(ans);
```

Backticks – it evaluates the string or format the string.(by directly passing the variables in a string using ${})

```
let ans = `my name is ${var2} ${var3} thankyou`
console.log(ans);
```

Number

Let num1 = 10;

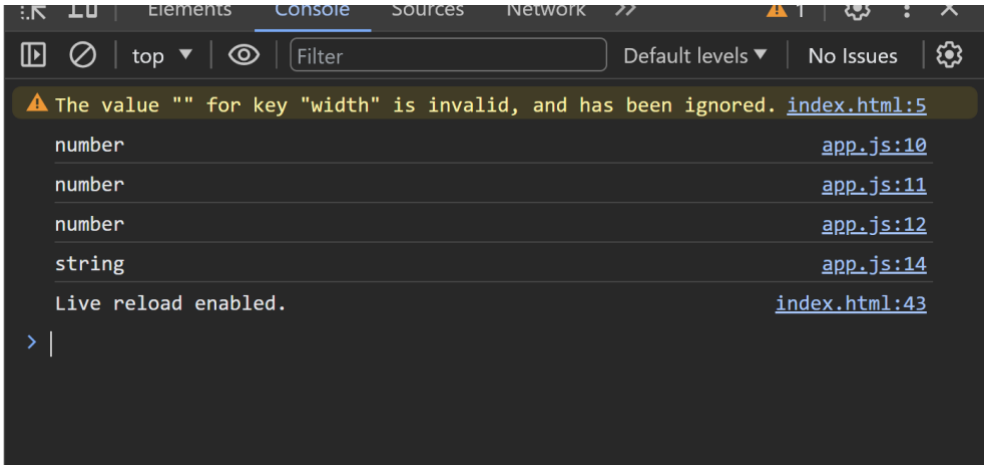
Let num2 = 10.75;

Let num3 = -199;

Type of (method):

Syntax – typeof(var or data type)

It is used to find the data type of the given variable.



Operators

- **Addition –**

```
let ans1 = num1 + num2;
```

- **Subtraction –**

```
let ans1 = num1 - num2;
```

- **Multiplication –**

```
let ans1 = num1 * num2;
```

- **Division –**

```
let ans1 = num1 / num2;
```

- **Remainder –**

```
let ans1 = num1 % num2;
```

- **Exponential –**

```
let ans1 = num1 ** num2;
```

Precedence rule of operators in JS

'PEMDAS'

P – Parenthesis
E – Exponential
M – Multiply
D – Division
A – Addition
S – Subtraction

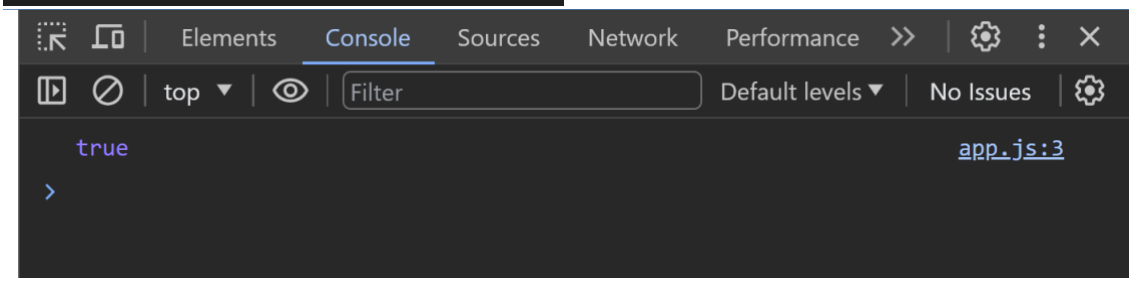
Boolean (T/F)

Two types of Boolean values are present there –

- True
- False

E.g. - Let ans1 = true ;

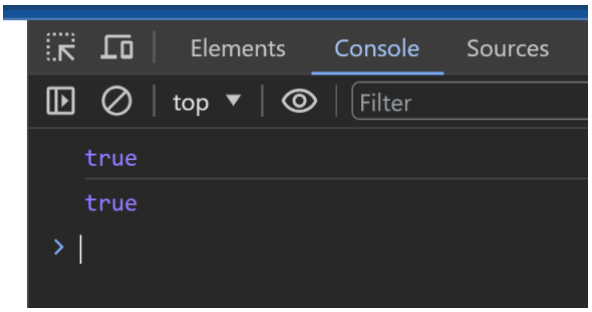
```
let sach = true;

console.log(sach);
```



Typecasting

- Typecasting means conversion of one data type to another data type.
- Typecasting is always done on the right side of the element.

```
// typecasting
let ans = sach == 1;
console.log(ans);
```



Types of Typecasting

There are two types of typecasting :-

- Implicit typecasting – internal type of typecasting or automatic type conversion done by interpreter.
- Explicit typecasting – type conversion which is done forcefully by the developer.

Comparison operator

1) General equality (==)

- This type of comparison is known as normal comparison.
- In this case, data types are not checked.

```
// typecasting
let ans = sach == 1;
console.log(ans);
```

In above example, it will return true as normal comparison does not check the data types, all the values other than 0 are considered as true, hence the result will be true.

2) Strict equality (===)

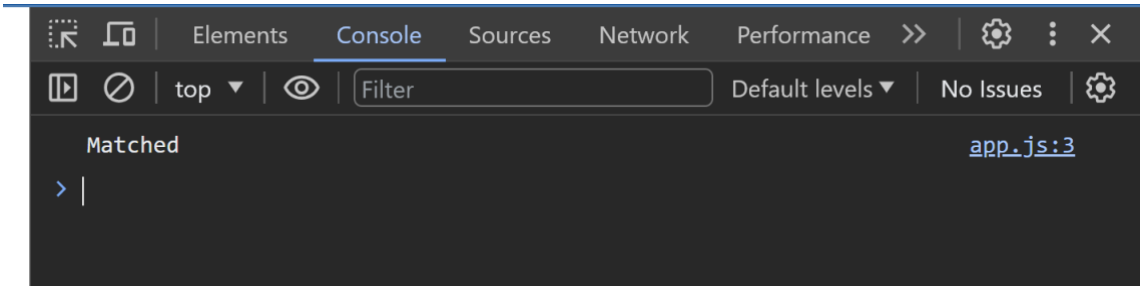
- Data types are compared.
- In this case, data types are checked.

```
let ans2 = sach === 1;
close.log(ans2);
```

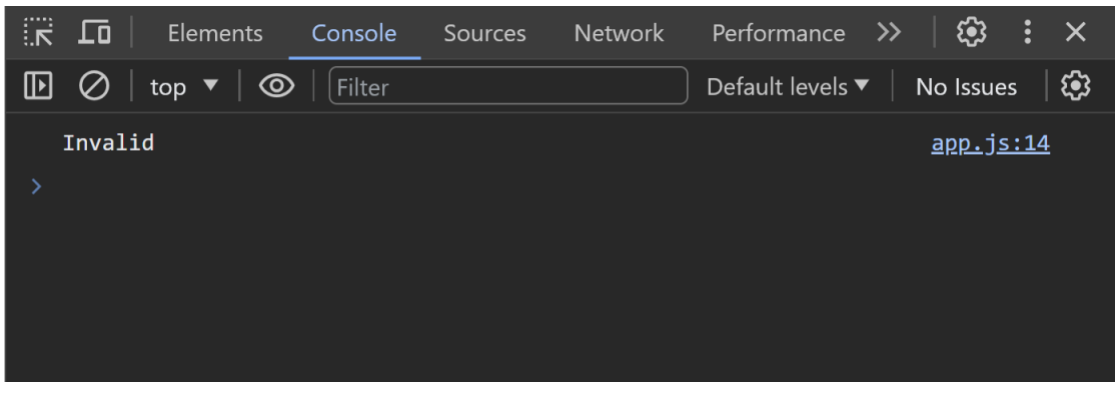
In above example, it will return false as this comparison operator checks the data type and since Boolean expression is not equal to a number, hence the result will be false.

Comparison

```
let age = 10;
if(age == '10'){
    console.log("Matched");
}
else{
    console.log("Invalid");
}
```

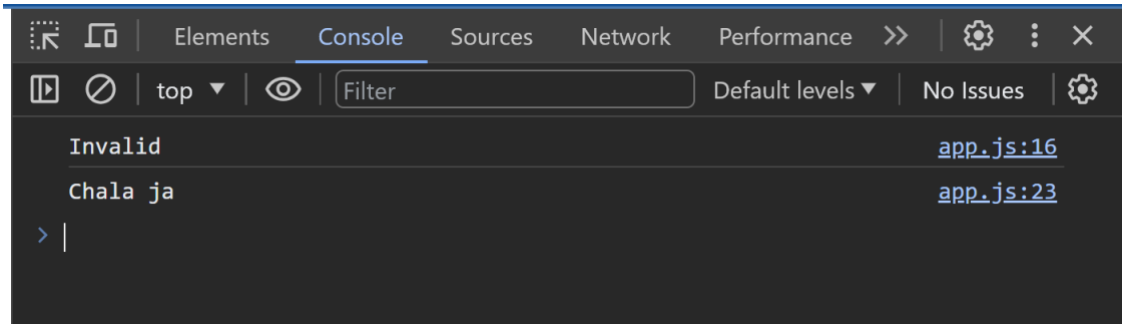



```
let age = 10;
if(age === '10'){
    console.log("Matched");
}
else{
    console.log("Invalid");
}
```



Practice ques-

```
let person = 24;
if(age < 18){
    console.log("Chala ja");
}
else if(age > 25){
    console.log("Le maze le");
}
else{
    console.log("Ishq no entry");
}
```



Short circuiting

```
let marks = 100;
let attendance = 88;
if(attendance>75 || mark>33){
    console.log('pass');
}
else{
    console.log('fail');
}
```

In above example, second condition(mark > 33) will not be evaluated as the first condition(attendance > 75) is true, thus no error will be generated. This is known as short circuiting.

Function

- It is a block of code that perform a specific task which can be reused.
- It does not work without calling.

Syntax – function <function name>(){ }

```
// function declaration
function diwali(){
    console.log("happy diwali ");
}


//function calling
diwali();
```

Types of Function

1) Simple function –

Basic type of function in which no parameters are passed.

```
function sum(){
    let a = 10;
    let b = 20;
    console.log(a+b);
}

sum();
```

2) Parameterized function -

There is no need of using let while defining parameters.

```
function sum2(a,b){
    console.log(a+b);
}

sum2(10,30);
```

Since the value of b is not defined thus we don't know the type of b, hence NaN is coming in the result.

NaN → Not a Number.

```
function kaju(a,b){
    console.log(a+b);
}
kaju(250);
```

```
NaN
```

3) Default Parameterized function –

In this type of function, default value is assigned to the parameter.

```
//default parameterised function
function kajuKatli(a,b=60){
    console.log(a+b);
}
kajuKatli(250);
```

Note – Value passed to the function is always dominant to default value. Thus, the variable will point to the passed value even if it has some default value.

```
function kajuKatli(a = 60,b){
    console.log(a);
    console.log(a+b);
}
kajuKatli(250);
```

```
250
NaN
```

Browser has two engines –

- **Layout engine** – see html and css
- **JS engine** – see JS

```
//undefined
function sam(a){
    let kaju;
    console.log(kaju);
    console.log(a + kaju);
}

sam(30);
```

```
undefined
NaN
```

JS engine will assign undefined to the variable itself.

```
//case 1
let kaju;
console.log(kaju);
```

Since the JS engine assigns undefined to the variable, it is not desirable to assign undefined value to any variable itself.

```
//case 2 --> we dont do this in real
let kaju = undefined;
console.log(kaju);
```

If we want our variable to point no value or nothing then inspite of assigning undefined to it we assign it null value.

Type of null is object.

```
//case 3
let kaju = null;
console.log(kaju);
```

Object

- Collection of properties.
- It is an unordered data structure.

Linear = ordered.

Nonlinear = unordered

Defining an object:

```
let person = {};
```

Properties: it is a pair of key value pair separated by semicolon.

```
let person = {
    naam : "Kaju",
    age : 1,
    favColor : 'red'
};
```

Printing object

```
let person = {
    // properties -> key : value
    naam : "kaaju lalla" ,
    umar : 1 ,
    pasandeedaRang : 'laal'
};
console.log(person);
```

```
▼ {naam: 'kaaju lalla', umar: 1, pasandeedaRang: 'laal'} i
    naam: "kaaju lalla"
    pasandeedaRang: "laal"
    umar: 1
  ▶ [[Prototype]]: Object
```

Since properties are not printing in the given sequence, thus object is a nonlinear data structure

Dot notation

To access a particular property of an object, we use dot notation.

```
let person = {
    // properties -> key : value
    naam : "kaaju lalla" ,
    umar : 1 ,
    pasandeedaRang : 'laal'
};
console.log(person);
//dot notation
console.log(person.naam);
```

```
▼ {naam: 'kaaju lalla', umar: 1, pasandeedaRang: 'laal'}                app.js:7
    naam: "kaaju lalla"
    pasandeedaRang: "laal"
    umar: 1
  ▶ [[Prototype]]: Object
kaaju lalla                                                             app.js:9
>
```

If we are accessing a function from an object then it needs to be called while accessing.

```
let person = {
    // properties -> key : value
    naam : "kaaju lalla" ,
    umar : 1 ,
    pasandeedaRang : 'laal' ,
    wishDiwali : function kismish(){
        console.log("appy deewaliiii");
    }
};
// console.log(person.wishDiwali); // wrong
console.log(person.wishDiwali()); // right
```

```
appy deewaliiii
undefined
>
```

Whenever a function is defined inside an object, it is called as method.

Array

- It is an ordered data structure.
- It is heterogeneous in nature.

Syntax -> let <array name> = [elements ……]

1D Array:

Ex – let arr = [ ];

Let arr = [10,20,30]; → homogeneous

Let arr = [10,"Sam",true,false,20.4,undefined] → heterogeneous

2D Array:

Let arr2 = [10,20,[100,200],40];

3D Array:

let arr3 = [10,20,[100,200,[6000],0],"sam"];

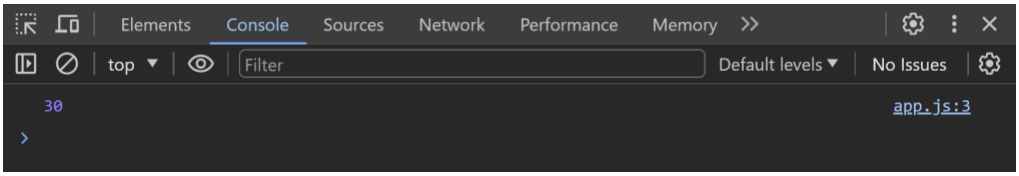
Accessing elements of an array:

<array name>[index]….

- arr[3]; {it will fetch the value of 3rd index, i.e., false.}
- arr2[2][0];
- arr3[2][2][0];

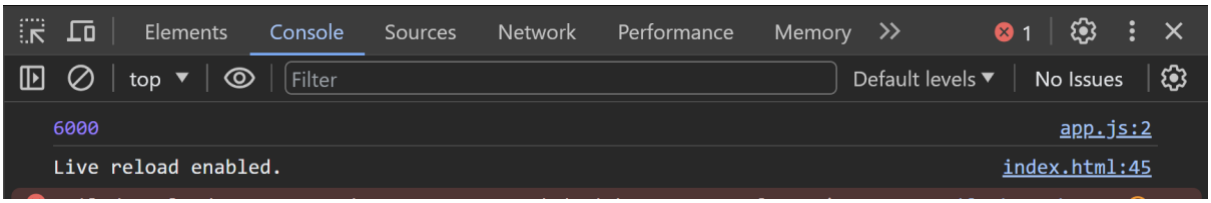
- **from 1D Array -**

```
let arr = [10,20,30,40];
console.log(arr[2]);
```



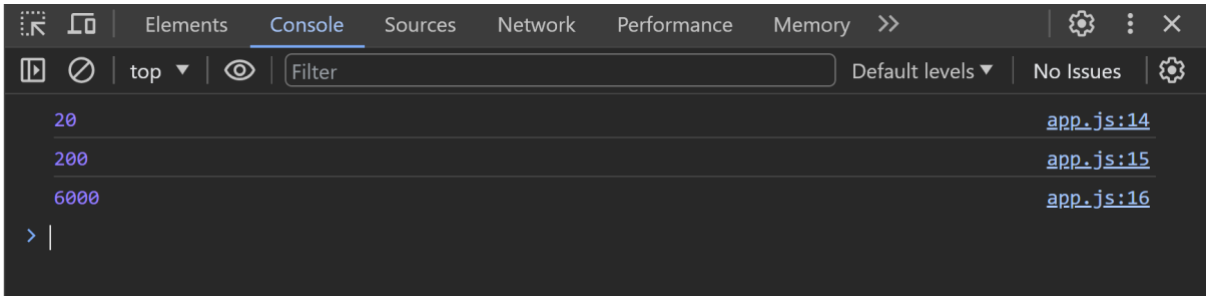
- **from 2D Array –**

```
let arr2 = [100,200,[6000],0];
console.log(arr2[2][0]);
```



- **from 3D Array –**

```
let arr3 = [10,20,[100,200,[6000],0],"sam"];
console.log(arr3[1]);
console.log(arr3[2][1]);
console.log(arr3[2][2][0]);
```

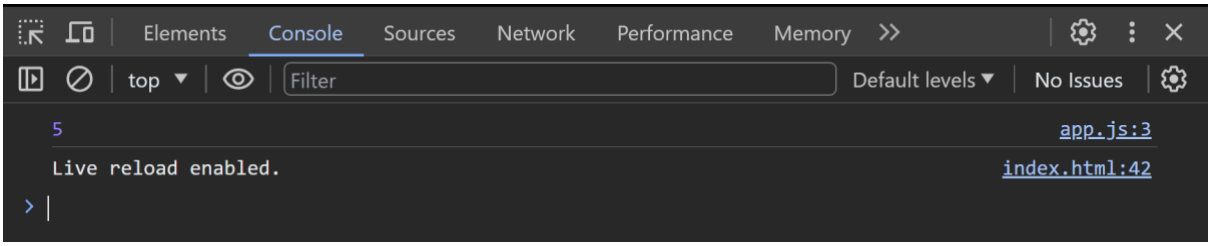


Array Methods

```
let arr = [10,20,30,40,50];
```

1. **Length** – it will return the length of the array.

```
console.log(arr.length);
```



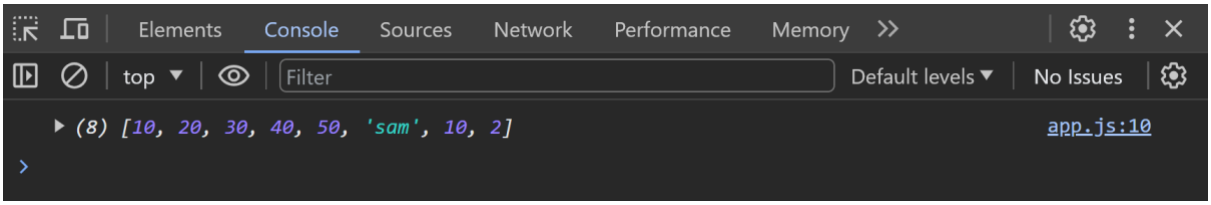
2. **Push** – it will add the given element at the end of the array and return type of push method is length of array.

```
let returnType = arr.push("sam");
console.log(returnType);
console.log(arr);
```

```
6
▶ (6) [10, 20, 30, 40, 50, 'sam']
>
```

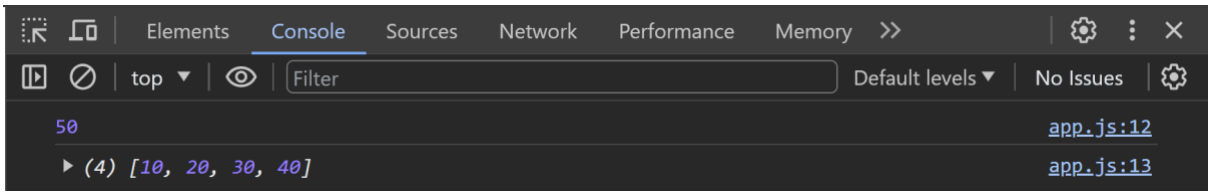
We can add multiple elements together using push.

```
let returnType = arr.push("sam",10,2);
console.log(arr);
```



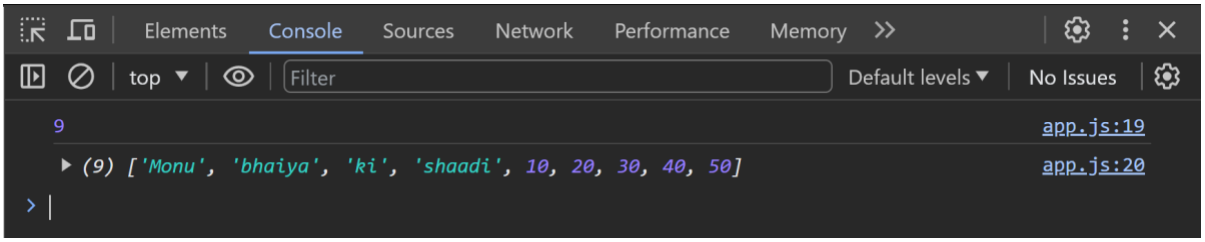
3. **Pop** – it will remove the last element of array and return type of pop method is removed element

```
let returnType = arr.pop();
console.log(returnType);
console.log(arr);
```



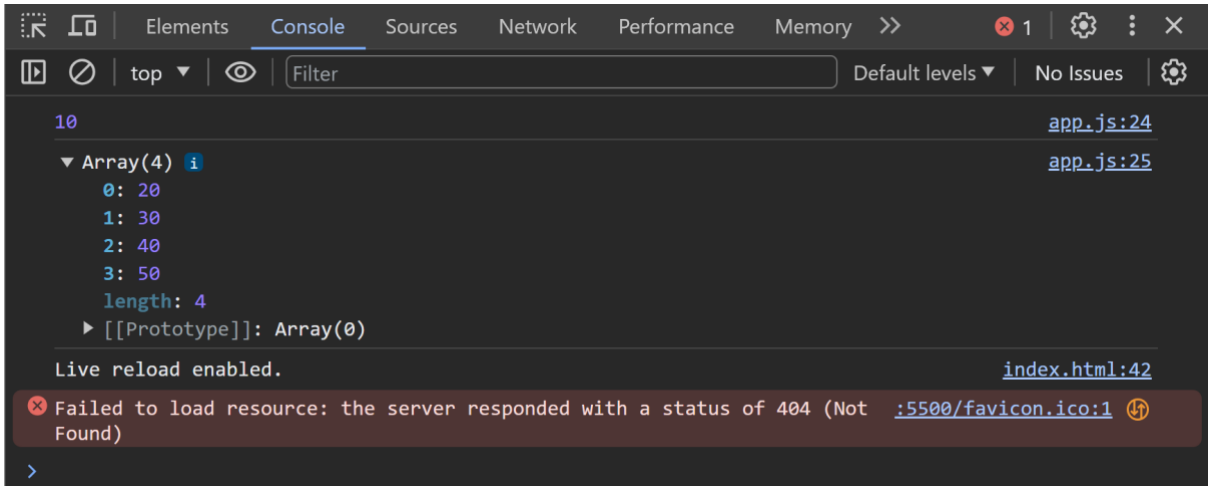
4. **Unshift –** it will add the given elements in the beginning of array and return type of this method is length of array.

```
let returnType = arr.unshift("Monu","bhaiya","ki","shaadi");
console.log(returnType);
console.log(arr);
```



5. **Shift –** it will remove the element from the beginning of array and return type of this method is removed element.

```
// 5. shift
let returnType = arr.shift();
console.log(returnType);
console.log(arr);
```



6. forEach –

Syntax - <Array name>.forEach(function(){})

It only iterates and does not return anything.

```
let arr = [10,20,30,40];

arr.forEach(function(item,index){
    console.log(`index: ${index} -- item: ${item}`);
})
```

```
index: 0 -- item: 10                    app.js:5
index: 1 -- item: 20                    app.js:5
index: 2 -- item: 30                    app.js:5
index: 3 -- item: 40                    app.js:5
Live reload enabled.                    index.html:42
```

7. Map –

Syntax - <Array name>.map(function(){})

It returns a new array and the length of new array depends on previous one.

```
let arr = [10,20,30,40,50];

let newArr = arr.map(function(item,index){
                return item * 2;
            })

console.log(arr);
console.log(newArr);
```

```
▸ (5) [10, 20, 30, 40, 50]              app.js:16
▸ (5) [20, 40, 60, 80, 100]             app.js:17
›
```

8. Filter –

Syntax – <Array Name>.filter(function(){})

It returns a new array whose length varies from 0 to the length the previous array or is independent of previous array.

```
let arr = [10,20,30,40,50];

let newArr = arr.filter(function(item,index){
    if(item >= 50){
        return true
    }
})

console.log(arr);
console.log(newArr);
```

```
▸ (5) [10, 20, 30, 40, 50]                    app.js:29
▸ [50]                                        app.js:30
›
```

9. Sort –

Syntax - <Array Name>.sort()

```
let arr = [10,40,20,50];

let out = arr.sort();

console.log(arr);
console.log(out);
```

```
▸ (4) [10, 20, 40, 50]                        app.js:38
▸ (4) [10, 20, 40, 50]                        app.js:39
›
```

It returns the sorted array.

10. **Find –**
11. **Reduce –**

Loops

Types of loops:

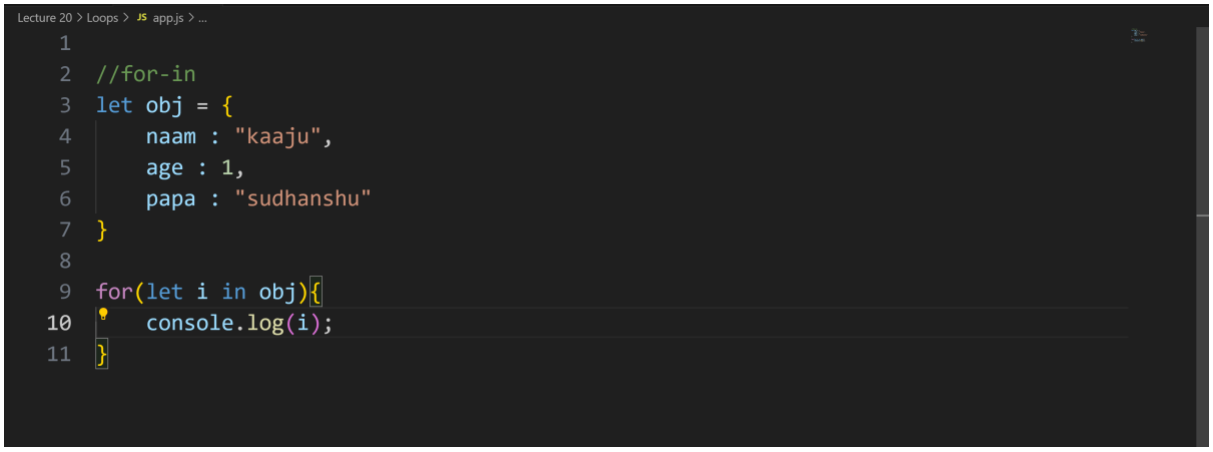
- for loop
- while loop
- do while loop
- for-in loop
- for-of loop

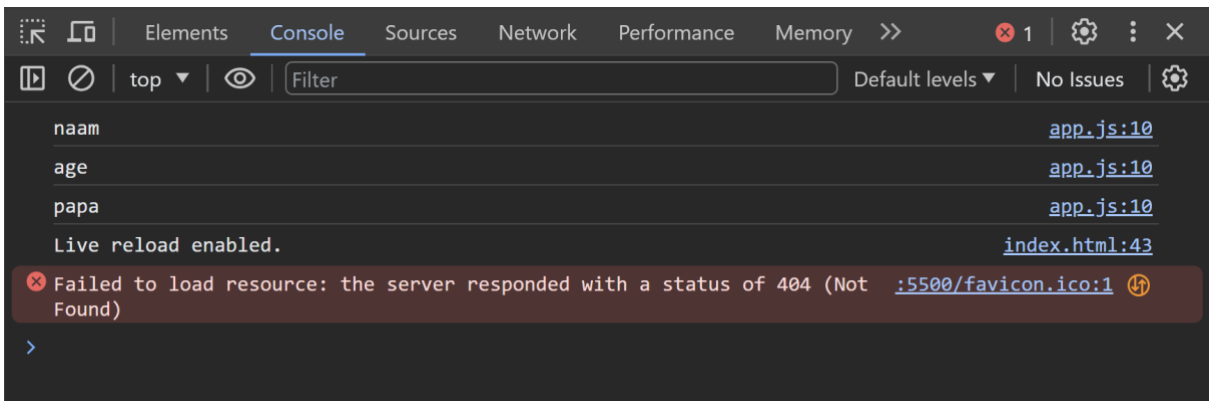
for-in loop -> works on objects.

for-of loop –> works on iterable items(arrays).

for-in loop –

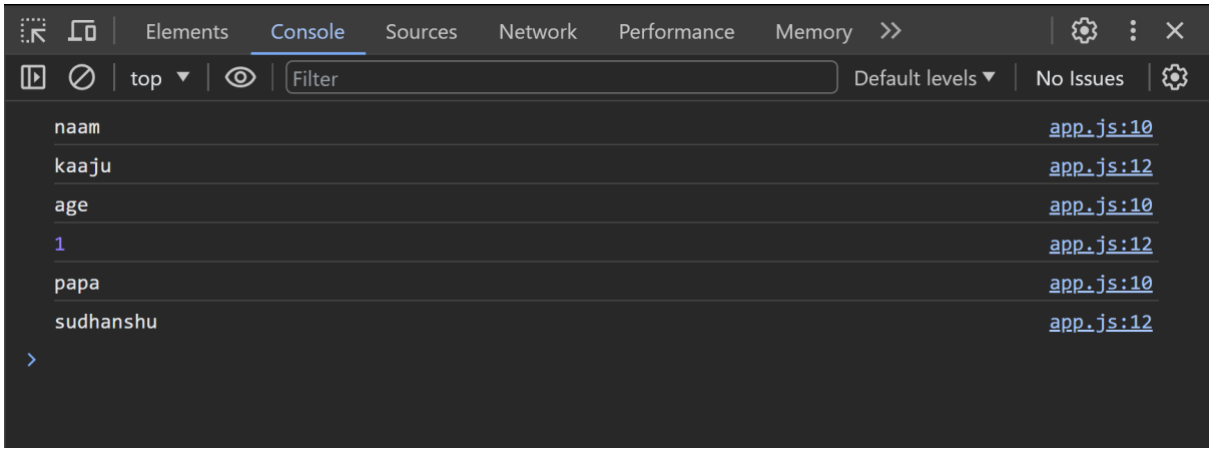
Syntax – for(declarative iterator in object){}





To access the value inside a loop, key is passed as an array method.

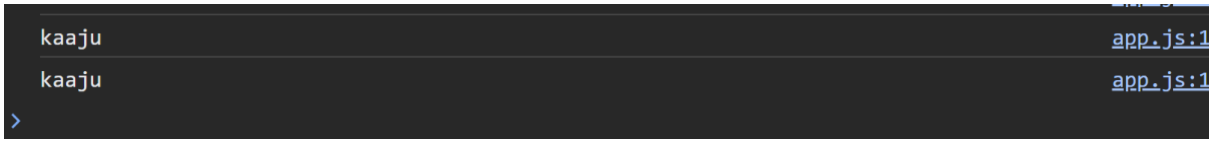
```
//for-in
let obj = {
    naam : "kaaju",
    age : 1,
    papa : "sudhanshu"
}

for(let i in obj){
    console.log(i);
    // console.log(obj.i); wrong
    console.log(obj[i]);
}
```



Note:

Keys are stored as strings. So if keys are passed as an array method outside the loop, key should be passed in string form.

```
console.log(obj.naam);
//console.log(obj[naam]);
console.log(obj["naam"]);
```



for-of loop

Syntax - for(declarative iterator of iterable item){}

```
// for-of()

let array = [10,20,true,30,40];

for(let item of array){
    console.log(item);
}
```